

Sanor 0.3.0

Precise animated presentation utility

<https://github.com/pacaunt/sanor>

@pacaunt

Documentation

2026-09-12

Sanor is a Thai word meaning ‘to present’.

This package aims to provide high flexibility in creating animated, rich pdf presentation from a small set of simple primitives, that are compatible for any Typst environment.

Contents

1	Introduction	2
2	Usage	3
2.1	Initialization	3
2.2	Creating a slide	3
2.3	Modifying Elements	4
2.4	Rule Specification	5
2.5	Timeline and Pause	10
2.6	Objects and Cases	11
2.7	Utility	15
2.8	Configuration	15
2.9	Examples	16
3	References	22
	Sanor Module	22
	Rules Module	27
	Object-case Module	31
	Components Module	32
	Pdfpc Module	34

1 Introduction

PDF presentation has been there for many decades; it is one of the most basic file that can be shared across platform, and almost all computer can display it. However, pdf cannot display moving components, as it is usually be able to in Powerpoint or other programs.

Therefore, the *animated* pdf presentation means a set of slides that made from **multiple pdf pages**, each may contain some slight differences, such that when being displayed page-by-page, the elements on it evolve *as if* they are animated.

There are many good Typst presentation packages out there, so what is the point of using Sanor, or me creating Sanor? Usually, those packages provide convenient interface and functionality for creating simple animated presentation such as `#pause`, `#uncover`, `#only`, or `#alternatives`. Those functions are powerful, it can create limitless sequence of animations. However, it is usually limited to using with `#content` type, or cannot be used in `#context` environment, because the management of states that control the animation steps is usually done by *parsing* and inspecting the content, before modifying it and creating slides.

This package uses a different philosophy: all animations are based on functions that are *pure* and can return *any* datatypes. Therefore, technically, Sanor can be used with any datatypes from any packages, getting rid of datatype conflict from package integration entirely.

Moreover, Sanor only have `#pause()` as a function, but all of other functionalities can be obtained through the `#tag()` and **animation rule specification**. So, this package allows user to define their own animation sequence, together with how it will evolve on each step.

A ‘slide’ consists of multiple frames of animation called ‘subslides’. Each element that user wants to make it evolve on each stepping subslide must be covered by using the `#tag(..)` function. User must name that element so that the engine knows which element needs to be modify. Then, user can specify the ‘rule’ telling what to do with the element (called ‘case’). The rule is then push into the animation sequence by `#s.push(rule)`. The modification specified by the rules will be present on each step in the same order according to the `#s` sequence.

With this method, the elements can be hidden, displayed, or modified in limitless number of ways, replacing all conventional pdf animation functions entirely.

2 Usage

Let's recall the components in an animated presentation. A 'slide' consists of multiple animation steps shown in each frame called 'subslide'. An element can be tagged and named, and modified by 'rules', containing descriptions of how to change its appearance called 'cases'.

2.1 Initialization

Import this package, and style the page and text as you want. In this tutorial's output, the configuration is as follows:

```
1 #import "@preview/sanor:0.3.0": *
2 // Styling
3 #set page(paper: "presentation-16-9")
4 #set text(size: 25pt)
5 // Your content
```

2.2 Creating a slide

There are two methods:

1. Use the `#slide()` function. This is recommended when the `#pause()` and `#tag()` functions are used together, and you need precise control of the animation sequence.
2. Use the `#scene()` function. This is optimized to only be used with `#tag()` function. The syntax is less verbose, but you cannot control the animation sequence freely like the `#slide()` function.

The templates of those functions are as shown:

`#slide()`

```
1 #slide(s => ([
2   // Shorthand to avoid repeating `s`
3   #let tag = tag.with(s)
4   // your content goes here
5 ], s))
```

`#scene()`

```
1 #scene(tag => [
2   // your content
3 ], controls: (
4   // your animation rules
5 ))
```

- The `#slide()` function accepts a function that returns *an array* of `([BODY], s)`, where `[BODY]` is the content you want to display on the slide, and `s` is the *updated* state containing the slide's animation sequence, and **is a required** argument for both 'pause' and 'tag' as `#pause(s, ..)` and `#tag(s, ..)`. To avoid repeating `s` in `#tag(s, ..)`, we usually write `#let tag = tag.with(s)` at the beginning of the slide and use this function like `#tag(..)` directly.
- The `#scene()` function accepts a function that returns a content directly, in the form of `tag => body`. This 'tag' is already contains the `#s` state, so **the `s` is not required** as argument. The tag is used as `#tag(..)` directly, and thus **`#pause(..)` cannot be used in `#scene()`**.

Here is a minimal example of how to use both functions to display the same animation:

`#slide()`

```
1 #slide(s => ([
2   #let tag = tag.with(s)
3   #tag("name", [Element])
4   // Update here
5   #s.push(apply("name"))
6 ], s))
```

`#scene()`

```
1 #scene(tag => [
2   #tag("name", [Element])
3 ], controls: (
4   apply("name"), // Update here
5 ))
```

In this example, `#apply("name")` is called a *rule*. A *rule* is a statement that tells what to do on which tagged element. In this case, the `#apply("name")` means to ‘apply’ (show) the element named `"name"` on the slide. In the `#slide()` function, the rule is specified in order by pushing it into `#s`, while in the `#scene()` function, it is specified in the `controls` argument instead.

The following sections will use the `#slide()`-like syntax with the shorthand

```
1 #let tag = tag.with(s)
```

at the beginning of the slide, as it is more explicit and flexible. Please make sure to include such line of code because the default `tag` function **requires** the animation sequence `s` as the first positional argument.

2.3 Modifying Elements

Elements that can be modified on the slide must be tagged and named, as

```
1 #tag(s, "name", [Element]) // or
2 #let tag = tag.with(s)     // Shorthand applied
3 #tag("name", [Element])
```

Then, it can be modified by specifying some rules. A rule has the following anatomy

```
1 rule("name", ..cases)
```

where the `cases` means the modifiers that will apply on the element named `"name"` in the subslides and alter its visibility according to the ‘rule’. The modifiers can be many things, the simplest ones are

- functions to wrap the element, or
- another element to replace the original tagged element.

For example, `apply("name", text.with(fill: red))` makes the text in the element `"name"` become red from this and the subsequent subslides. Normally, the tagged elements are hidden until the rules that can reveal the element are applied. The `apply("name")` also can be used to make the element `"name"` become visible. Moreover, the `apply` rule can be used to ‘apply’ more modifiers to the element in the next steps, for example,

```
1 #s.push(apply("name"))           // shows [Element]
2 #s.push(apply("name", text.with(fill: red))) // shows red [Element]
3 #s.push(apply("name", strong))    // shows red and bold [Element]
```

The full example of this slide would be

```
1 #slide(s => ([
2   #let tag = tag.with(s) // Shorthand to avoid repeating `s`.
3   #set text(size: 100pt) // for visually apparent images
4   #tag("name", [Element])
5   #s.push(apply("name"))           // on subslide 1
6   #s.push(apply("name", text.with(fill: red))) // on subslide 2
7   #s.push(apply("name", strong))    // on subslide 3
8 ], s))
```



2.4 Rule Specification

A Quick Tutorial

Let's say we want to hide an element `"name"` first, then display it after, make it red, emphasize it once, and revert it to its base form. These visible states can be specified by using *rules*. For this example, the rules that involved are

- `cover("name")` makes an element hidden in this and subsequent steps,
- `apply("name", text.with(fill: red))` for making its text red,
- `once("name", emph)` applies the `#emph()` function to the element once,
- `revert("name")` reverts the element back to its base form, before applying anything.

After creating a new slide from the template and tagging the element, we now should have

```
1 #slide(s => ([
2   #let tag = tag.with(s)
3   #tag("name", [Element])
4   // We will specify the rule here
5 ], s))
```

Actually, tagged elements are hidden by default, but for this tutorial, we will make the element hidden by specifying an explicit rule. To do this, the `cover("name")` is added in the animation sequence `#s`:

```
1 #slide(s => ([
2   #let tag = tag.with(s)
3   #tag("name", [Element])
4   // We will specify the rule here
5   #s.push(cover("name"))
6 ], s))
```

Now, the element `"name"` is hidden on the first subslide. Then, to show the element on the next slide, just push the `apply("name")` next, as

```
1 #slide(s => ([
2   #let tag = tag.with(s)
3   #tag("name", [Element])
4   // We will specify the rule here
5   #s.push(cover("name"))
6   #s.push(apply("name")) // <-- make it shown
7 ], s))
```

This rule creates a new subslide, with the element shown. The other steps' modification to the element are specified in the same manner, so you should have something like (stripping the slide environment)

```
1 #s.push(cover("name"))           // on subslide 1
2 #s.push(apply("name")) // display it // on subslide 2
```

```

3 #s.push(apply("name", text.with(fill: red))) // on subslide 3
4 #s.push(once("name", emph))                // on subslide 4
5 #s.push(revert("name"))                     // on subslide 5

```

However, let's add a *blank* animation, i.e., a subslide that we do not specify any rule on it, but let the animation sequence happens on its own. In the algorithm, this rule will just add a new subslide, but since `once("name", emph)` rule applies only once, adding this blank subslide will make the result clear that the modification from `once("name", emph)` is gone. We can do this by pushing an empty array to the sequence as

```

1 #s.push(cover("name"))                      // on subslide 1
2 #s.push(apply("name")) // display it       // on subslide 2
3 #s.push(apply("name", text.with(fill: red))) // on subslide 3
4 #s.push(once("name", emph))                // on subslide 4
5 // Do nothing, to see that the once's effect is gone
6 #s.push(())                               // on subslide 5
7 #s.push(revert("name"))                   // on subslide 6

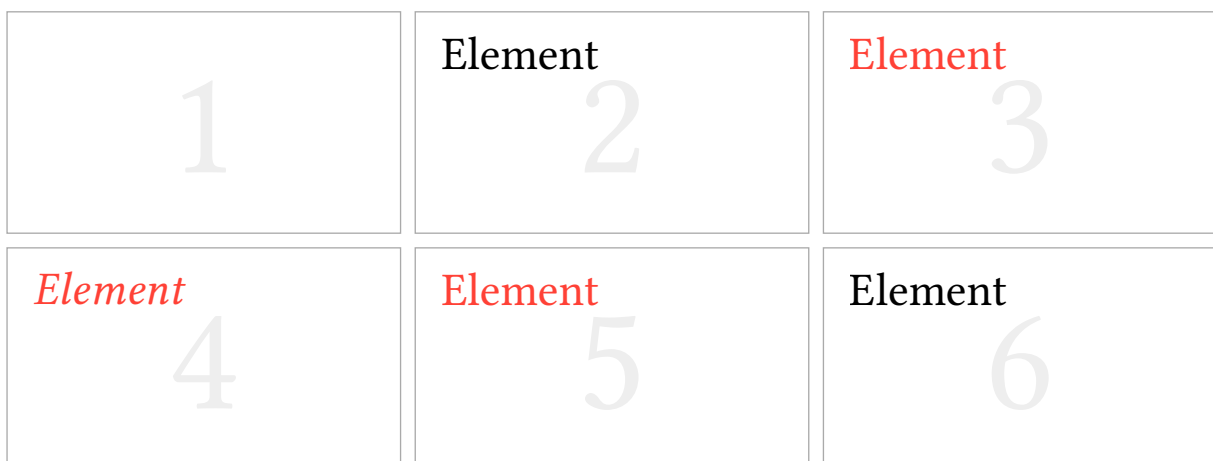
```

The full example of this usage is shown here

```

1 #slide(s => ([
2   #let tag = tag.with(s)
3   #set text(size: 100pt) // for visually apparent images
4   #tag("name", [Element])
5   #s.push(cover("name"))                      // on subslide 1
6   #s.push(apply("name")) // display it       // on subslide 2
7   #s.push(apply("name", text.with(fill: red))) // on subslide 3
8   #s.push(once("name", emph))                // on subslide 4
9   // Do nothing, to see that the once's effect is gone
10  #s.push(())                               // on subslide 5
11  #s.push(revert("name"))                   // on subslide 6
12 ], s))

```



Properties of A Rule

The more proper explanation of rules' ability is the following. A rule can control the visible state of an element by three properties:

1. *active*: The rule that has `active: true` can make the element *visible*, `active: false` hide the element, and `active: auto` follows the visibility from the previous active rules.

2. *remain* : Control whether the modification remains in the subsequent subslides. `remain: true` means the modification remains.
3. *inherit* : Control whether to combine the current modification to the previous from the active rules. `inherit: true` means the modifications are comined.

These properties explain the role of the following rules:

rules	active	remain	inherit
<code>apply(name, ..cases)</code>	<code>true</code>	<code>true</code>	<code>true</code>
<code>once(name, ..cases)</code>	<code>true</code>	<code>false</code>	<code>true</code>
<code>cover(name, ..cases)</code>	<code>false</code>	<code>true</code>	<code>true</code>
<code>revert(name, ..cases)</code>	<code>auto</code>	<code>true</code>	<code>false</code>
<code>force(name, ..cases)</code>	<code>true</code>	<code>true</code>	<code>false</code>
<code>clear(name, ..cases)</code>	<code>auto</code>	<code>true</code>	<code>false</code>

These properties can be modified in all rule functions, except `#clear()`, by specifying `active: (bool)`, `remain: (bool)`, `inherit: (bool)` when calling them. However, some rules have their own special effects:

- `clear()` will clear *all* previous modifications done to the element.
- `revert()` will display the element in its base form, but the modification history is intact. Therefore, when the next `apply` or other rules that has `inherit: true`, the old modifications will come back.
- `once()` can make the element visible only once (and hidden thereafter), if it is the last active rule of that element.

The difference of using `revert("name")` and `clear("name")` are shown in the following examples:

```

1 #slide(s => ([
2   #let tag = tag.with(s)
3   #set text(size: 50pt) // for visually apparent images
4   = Revert Example
5   #tag("name", [Element])
6   #s.push(apply("name"))
7   #s.push(apply("name", rect.with(inset: 0.5em)))
8   #s.push(apply("name", text.with(fill: red)))
9   #s.push(revert("name")) // <-- base form.
10  #s.push(apply("name")) // <-- come back!
11 ], s))

```

Revert Example

Element

1

Revert Example

Element

2

Revert Example

Element

3

Revert Example

Element

4

Revert Example

Element

5

So, `revert("name")` **does not clear** the modification history, as on the subslide 5 the modified element came back just like the one before the `revert` call. Let's compare this to the `clear` rule:

```
1 #slide(s => ([
2   #let tag = tag.with(s)
3   #set text(size: 50pt) // for visually apparent images
4   = Clear Example
5   #tag("name", [Element])
6   #s.push(apply("name"))
7   #s.push(apply("name", rect.with(inset: 0.5em)))
8   #s.push(apply("name", text.with(fill: red)))
9   #s.push(clear("name")) // <-- base form.
10  #s.push(apply("name")) // <-- No coming back...
11 ], s))
```

The element after `clear("name")` stays like the base form. It does not change back to the modified one displayed in the subslide 3, as `clear` does clear the modification history of the element.

Same Tag – Same Fate

Tagged elements in the same slide that have the same name will transform in the same way by the same rules. For example, let's say we want to show simplifying the equation

$$\frac{x(x+1)}{2(x+1)}$$

by showing the common parts $(x+1)$ as

- red text in the subslide 2,
- cancel text in the subslide 3,
- eliminate text in the subslide 4

Since the $(x+1)$ will undergo the same process, we can tag them with the same name and specify the rules with that name, like this

```
1 #slide(s => ([
2   #let tag = tag.with(s)
3   #set text(size: 50pt) // for visually apparent images
4   = Math Simplification
5   // same tag name "expr"
6   $ (x tag("expr", (x + 1))) / (2 tag("expr", (x + 1))) $
```

```

7  #s.push(apply("expr")) // to show the element
8  #s.push(apply("expr", text.with(fill: red))) // make it red
9  #s.push(apply("expr", math.cancel.with(stroke: red))) // cancel it
10 #s.push(apply("expr", it => none)) // eliminates it
11 ], s))

```

Math Simplification

$$\frac{x(x+1)}{2(x+1)}$$

Math Simplification

$$\frac{x(x+1)}{2(x+1)}$$

Math Simplification

$$\frac{x\cancel{(x+1)}}{2\cancel{(x+1)}}$$

Math Simplification

$$\frac{x}{2}$$

Specifying Multiple Rules

On a slide, you may have many elements that should be transformed together in some steps. Multiple rules in the same step can be specified by using *an array of rules* instead. Let's say we want to make the following animation

Two Rects



Two Rects



First, initialize the slide and create the rectangle by

```

1  #slide(s => ([
2    #let tag = tag.with(s)
3    = Two Rects
4    #grid(
5      columns: (1fr, 1fr), align: center,
6      tag("rect1", rect(width: 2in, height: 1in)),
7      tag("rect2", rect(width: 2in, height: 1in)),
8    )
9  ], s))

```

Then, to show the rectangles, the `apply` rules are combined in an array and pushed into `#s` as

```

1  #s.push((apply("rect1"), apply("rect2")))

```

Similarly, the red and blue rectangles are also shown by replacing the old ones with another `apply` rule that contains the new rectangles. The second subslide are obtained by adding this line:

```

1  #s.push((

```

```

2  apply("rect1", rect(width: 2in, height: 1in, fill: red)),
3  apply("rect2", rect(width: 2in, height: 1in, fill: blue))
4  ))

```

Therefore, multiple rules can be executed in the same step by **combining them in an array** before pushing it to the sequence `#s`.

The full code of this example is shown here:

```

1  #slide(s => ([
2    #let tag = tag.with(s)
3    #set text(size: 50pt) // for visually apparent images
4    = Two Rects
5    #grid(
6      columns: (1fr, 1fr), align: center,
7      tag("rect1", rect(width: 2in, height: 1in)),
8      tag("rect2", rect(width: 2in, height: 1in))
9    )
10   // Show both rectangle together
11   #s.push((apply("rect1"), apply("rect2")))
12   // Change them to another in the same step
13   #s.push((
14     apply("rect1", rect(width: 2in, height: 1in, fill: red)),
15     apply("rect2", rect(width: 2in, height: 1in, fill: blue))
16   ))
17 ], s))

```

Later in this documentation, the method that can actually modify the properties of tagged elements will be introduced, so that the color of the rectangles can be changed without creating new ones.

2.5 Timeline and Pause

The animation sequence `#s` contains the sequence of rules, and also can contain the information about the timeline of the animation. In a basic sequence that only contains the rules, each step will create one subslide except for the first step which will be shown in the first subslide (not creating a new one).

Apart from pushing `()` to create an empty rule that forces one subslide, you can pushing *integers* like `1`, `2`, to create that number of subslides. For example

```

1  #s.push(once("A"))
2  #s.push(2) // shift the next subslide to execute the next rule
3  #s.push(once("B"))

```

will cause the element `"A"` to be visible on subslide 1, and element `"B"` to be visible on subslide 4 (2, and 3 are *skipped* by the `s.push(2)`). This is useful for creating step-by-step reveal with `#pause()` function, like this

```

1  #slide(s => ([
2    #s.push(1) // Skip the first subslide
3    #pause(s)[- First Point]
4    #s.push(1) // Creates the subslide 2
5    #pause(s)[- Second Point] // This will be shown in the subslide 3
6    #s.push(1) // Creates the subslide 3

```

```
7 ], s))
8
```

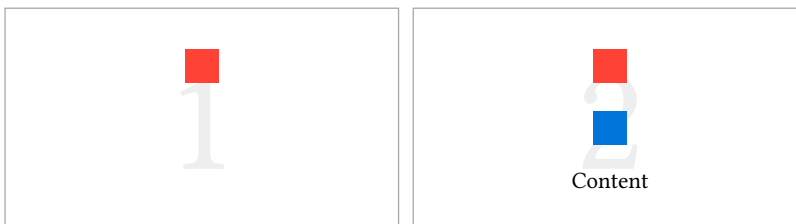


The showing sequence of contents that are tagged and paused will follow the sequence in `#s`. The integer skips the subslide and push the animation to the next subslide. Moreover, the *negative* integer also can be specified, to move the next rule/pause execute earlier than the current subslide, e.g.

```
1 #s.push(apply("A")) // shown on subslide 1
2 #s.push(apply("B")) // shown on subslide 2
3 #s.push(-1)          // shift the next animation to 1 subslide earlier
4 #pause(s)[Content]   // shown on subslide 2, instead of 3
5 #s.push(1)           // should create subslide 3, but got shifted to 2.
```

For a full example of this usage:

```
1 #slide(s => ([
2   #let tag = tag.with(s)
3   #set align(horizon + center)
4   #tag("A", rect(width: 1in, height: 1in, fill: red))
5   #tag("B", rect(width: 1in, height: 1in, fill: blue))
6   #s.push(apply("A")) // shown on subslide 1
7   #s.push(apply("B")) // shown on subslide 2
8   #s.push(-1)         // shift the next animation to 1 subslide earlier
9   #pause(s)[Content]   // shown on subslide 2, instead of 3
10  #s.push(1)           // should create subslide 3, but got shifted to 2.
11 ], s))
12
```



2.6 Objects and Cases

Animate with Objects

Objects are reusable element that can have multiple forms called *cases* and can be modified from the ground up. To create an object, use the `#object(func)` where `func` is any element function which will be used to create the display. For example,

```
1 #let my-rect = object(rect)
```

`my-rect` becomes a Sanor object that will use `rect` function to display. Objects cannot display directly on the slide, they have to be inside a `#tag()` function. So, `my-rect` needs to be displayed as

```
1 // Don't forget `#let tag = tag.with(s)` for `#slide(..)`.
2 #tag("name", my-rect())
```

The advantage of using object as element in the tag is the element can be modified way more flexible. The apparent thing that objects can do but other elements cannot is modifying the properties that must be specified as arguments like `stroke` and `fill`. To do this, you can write

```
1 #s.push(apply("name", fill: red))
```

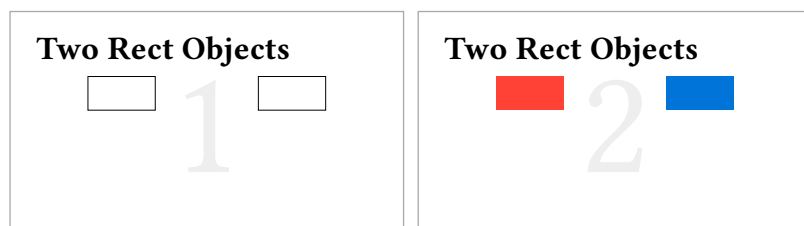
to make the `my-rect` red filled! Thus, **any keyword properties in the defined object can be modified through rules**. Revisiting the example of changing the rectangles' colors, we can write

```
1 #let my-rect = object(rect.with(width: 2in, height: 1in))
```

and use it as

```
1 #grid(
2   columns: (1fr, 1fr), align: center,
3   tag("rect1", my-rect()),
4   tag("rect2", my-rect())
5 )
6 #s.push((apply("rect1"), apply("rect2")))
7 #s.push((
8   apply("rect1", fill: red), // Change the fill directly!
9   apply("rect2", fill: blue),
10 ))
```

Note that the created object is used as if it is a normal element. You can add the styling arguments or positional argument as usual. From the snippet, the following result is obtained.



Advanced Modification: `#case()` function

The created object can also be modified by wrapping it with functions. To combine the keyword properties like `stroke: red`, `fill: green` and functions like `rotate()`, `align()`, the `#case()` function is used as

```
1 case(stroke: red, fill: green, rotate.with(45deg), align.with(center))
```

- The functions are positional arguments, and
- keyword properties are keyword arguments of the `case` function.

Then, the created case can be used in rules like

```
1 #s.push(apply(
```

```

2  "name",
3  case(stroke: red, fill: green, rotate.with(45deg), align.with(center)
4  ))

```

Predefined Cases

Some modification that are used very often can be named within the object/tag so that it can be applied by calling the case's name. Let's see this usage from the following example.

```

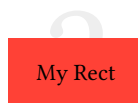
5  // Create 'my-rect' object, with predefined cases
6  #let my-rect = object(
7    rect.with(width: 4in, height: 2in),
8    redden: case(fill: red),
9    grayed: case(fill: gray),
10   bigger: scale.with(150%) // this is a shorthand for `case(scale.with(150%))`
11 )
12 // The content
13 #set align(center + horizon)
14 #tag("rect", my-rect[My Rect])
15 // Rules
16 #s.push(apply("rect"))
17 #s.push(once("rect", "redden")) // apply the 'redden' predefined case.

```

Predefined Case



Predefined Case



Predefined Case



There are some important cases that are predefined by default for all tagged elements:

1. **"hidden"** case: the modification when the element is hidden. It is some kind of `hide` function by default, but you can change it by redefining this in object initialization.
2. **"base"** case: no modification. This is the case that will be used whenever the object is display but no modification is specified.

Components – Object's Shortcut

Since the created object must be used with `tag` function, so why don't we combine them?—Here it is, the `#component` module. This module will let you create object that can be used directly in one function. You can use `#component.new(name, func)` to create a new component function, where `name` is the *default* name of the created (internal) object and `func` is the element function like initializing a new object. For example,

```

1  #let my-rect = component.new("rect", rect.with(fill: green))
2  // Change `s` to `tag` if you are using `#scene()`
3  #my-rect(s, [MY GREEN RECT])
4  #my-rect(s, [MY RED RECT], fill: green, name: "red-rect")

```

Note that all component will have `name` keyword argument for naming the object and being used internally by tag. The default is the one declared in the `component.new()`. Like objects, you can define cases and modify it with rules in the same way, e.g.

```

1 #s.push(apply("rect")) // shows the MY GREEN RECT
2 #s.push(apply("red-rect", rotate.with(45deg)))

```

Predefined Components

You can import the 'mcomps' module containing predefined markup components for drawing basic stuff like rectangle, circle, block, etc. These defined components have the same name as the standard Typst's but are prefixed with `m`, like `mrect` for `rect`, `mcircle` for `circle`, `mline` for `line`.

This package also contains predefined components from CeTZ package. All element drawing functions in the 'draw' module of CeTZ are available with prefix of `c` like `ccontent`, `ccircle`, `cline`, `crect`. The CeTZ elements are hidden by CeTZ's `hide` function by default, and **you can use the element's name** to refer in the rule specification too.

```

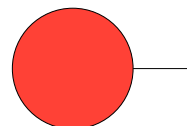
2 #import ccomps: * // Import the component
3
4 = CeTZ ccomps
5
6 #cetz.canvas({
7   import cetz.draw: *
8   ccircle(s, (0, 0))
9   // default name is 'c' + element's function
10  line("ccircle.east", (rel: (3, 0)), name: "l1")
11  // name of the CeTZ objects will be used as a name in tag too
12  crect(s, (6, -1), (8, 1), name: "r1")
13 })
14 #s.push("ccircle")
15 #s.push(apply("ccircle", fill: red, radius: 3))
16 #s.push("r1")
17 #s.push(apply("r1", stroke: green + 5pt))

```

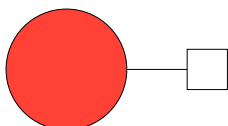
CeTZ ccomps



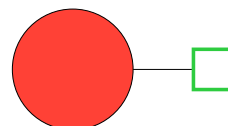
CeTZ ccomps



CeTZ ccomps



CeTZ ccomps



2.7 Utility

Magic selector

`select(element)` function accepts common elements that are created with element functions and generates a selector that is precise for styling specific element. For example,

```
1 #show select($a$): set text(fill: red)      a a
2 a // not selected
3 $a$ // selected
4
```

This usually works with ‘grouped’ elements like `$a^x$`, `$a_x$`, `$(..)$`, `block`, `rect` and so on. However, the ‘sequential’ elements like `$x + y$` do not work as expected, because Typst may collapse the sequence with another surrounding sequence making the element not detectable.

Ppfpic Integration

This package imports the Polylux’s pdfpc module and integrated within the slide function. To use this feature, please refer to their package: <https://polylux.dev/book/external/pdfpc.html>.

2.8 Configuration

Global Options & Handout Mode

You can set some default behavior of the functions by setting the slide options as

```
1 #let (slide,) = set-option(..options)
```

or

```
1 #slide(
2   options: (..options),
3   s => ([ /* body */], s)
4 )
```

The available options are

- `handout` default: `false`, the handout mode. By default, the handout slide will present only the last subslide. This can be changed by setting `handout-index` to the desired subslide.
- `handout-index` default: `auto` the index of the subslide used in the handout mode. Note that the first subslide’s index is `1` NOT `0`.
- `..new-options` the other configuration that will be passed to the slide/scene function.

Slide Options

The following keyword arguments can be passed to the slide/scene function to set some defaults.

- `defined-cases` accepts a dictionary in the form of ‘name: case’. The name of cases defined here can be referred when applying rules.
- `hidden` accepts a function or a case. This will be applied on the element when it is hidden by default. The default function is a modified `hide` that can hide lists and enums.
- `is-shown`: `false` Whether to display the tagged elements by default. If this is set to `false`, the tagged elements are hidden until any active rule is applied.

2.9 Examples

Syncing Animation

```
1 #import "@preview/sanor:0.3.0": *
2
3 // Set up presentation format.
4 #set page(paper: "presentation-16-9", fill: luma(20))
5 #set text(size: 25pt, fill: white)
6
7 #slide(s => (
8   [
9     #let tag = tag.with(s)
10    = Simultaneous Animation
11    #set align(center + horizon)
12
13    #grid(columns: (1fr,) * 2, align: horizon)[
14      #tag("Jack")[Jack]
15
16      #tag("jtext")[Jack was a teacher.]
17    ]
18    #tag("Julie")[Julie]
19
20    #tag("ltext")[Julie was a student.]
21  ]
22
23  #s.push((
24    apply("Jack"),
25    apply("Julie"),
26  ))
27
28  #s.push((
29    once("Jack", circle.with(fill: blue)),
30    once("jtext"),
31  ))
32
33  #s.push((
34    once("Julie", circle.with(fill: fuchsia)),
35    once("ltext"),
36  ))
37 ],
38 s,
39 ))
```

Simultaneous Animation

Jack

Julie

Simultaneous Animation

Jack

Julie

Jack was a teacher.

Simultaneous Animation

Jack

Julie

Julie was a student.

Math Animation

```
1 #import "@preview/sanor:0.3.0": *
2
3 // Set up presentation format.
4 #set page(paper: "presentation-16-9", fill: luma(20))
5 #set text(size: 25pt, fill: white)
6
7 #slide(s => (
8   [
9     #let tag = tag.with(s)
10    = Math Example
11    Simplify
12    $
13    (x(x + 1)tag("m", (x + 2)))/(2 tag("m", (x + 2)))
14    $
15    #s.push(apply("m")) // Show the `m`.
16    #s.push(apply("m", text.with(fill: red))) // Make it red.
17    #s.push(apply("m", math.cancel)) // Cancel it.
18    #s.push(apply("m", it => none)) // Remove it.
19    #s.push(-1) // to move the `pause` up
20    #pause(s)[End.]
21    #s.push(1) // to update the last `pause`.
22  ],
23  s,
24 ))
```

Math Example

Simplify

$$\frac{x(x+1)(x+2)}{2(x+2)}$$

Math Example

Simplify

$$\frac{x(x+1)(x+2)}{2(x+2)}$$

Math Example

Simplify

$$\frac{x(x+1)(x+2)}{2(x+2)}$$

Math Example

Simplify

$$\frac{x(x+1)}{2}$$

End.

Code Animation

Integration with Zebraw package.

```
1 #import "@preview/sanor:0.3.0": *
2
3 // Set up presentation format.
4 #set page(paper: "presentation-16-9", fill: luma(20))
5 #set text(size: 25pt, fill: white)
6
7 #slide(s => (
8   [
9     #import "@preview/zebraw:0.6.3": zebraw
10    #let tag = tag.with(s)
11    #let zebraw = zebraw.with(
12      background-color: luma(30),
13      lang: false,
14      highlight-color: white.transparentize(90%),
15      comment-color: white.transparentize(90%),
16      comment-font-args: (font: "Libertinus serif"),
17    )
18    = Code Example
19    Integration with `zebraw`.
20    #set align(horizon)
21
22    #show: zebraw
23
24    #tag("snippet")["`typst
25    #slide(s => ([
26      #let tag = tag.with(s)
27      // Your Content Goes Here
28    ],s))
29    ```]
30
31    #s.push(apply("snippet"))
32    #s.push(apply("snippet", it => {
33      zebraw(it, highlight-lines: (
34        "2": [
35          Don't forget this line!
36        ],
37      ))
38    })))
39
40 ],
```

```
41 s,
42 ))
```

Code Example

Integration with zebraw.

```
1 #slide(s => ([
2   #let tag = tag.with(s)
3   // Your Content Goes Here
4 ],s))
```

Code Example

Integration with zebraw.

```
1 #slide(s => ([
2   #let tag = tag.with(s)
3   > Don't forget this line!
4   // Your Content Goes Here
5 ],s))
```

Multiple Objects & Cases

Using rules and pauses together

```
1 #import "@local/sanor:0.3.0": *
2
3 // Set up presentation format.
4 #set page(paper: "presentation-16-9", fill: luma(20))
5 #set text(size: 25pt, fill: white)
6
7 #slide(
8   defined-cases: (
9     "highlighted": case(block.with(outset: 0.5em, stroke: yellow)),
10    "alert": case(text.with(fill: yellow)),
11  ),
12  s => (
13    [
14      #let tag = tag.with(s)
15      = Mixing animations and pause
16      // Go to the next subslide.
17      #s.push(1)
18      // Put the first pause
19      #pause(s, [Hello, there!])
20      #s.push(1) // then go to the next slide.
21
22      // This will be shown with the next animations.
23      #pause(s, [Here are some switches you can choose.])
24      #s.push(1) // then go to the next subslide.
25
26      #set align(center + horizon)
27
28      #let myrect = object(rect.with(height: 2cm, width: 2cm))
29
30      #grid(columns: (1fr,) * 2, align: top)[
31        #tag("green", myrect(fill: green))
32        #tag("gtext", [If you press the green one, nothing will happen.])
33      ][
34        #tag("red", myrect(fill: red))
35        #tag("rtext", [If you press the red one, also nothing...])
36      ]
37    ]
38  )
39)
```

```

38     #s.push((
39         apply("green"),
40         apply("red"),
41     ))
42     #s.push((
43         once("gtext", "alert"),
44         once("green", "highlighted"),
45     ))
46     #s.push((
47         once("rtext", "alert"),
48         once("red", "highlighted"),
49     ))
50
51     #pause(s, move(dy: -2em)[What did you choose?])
52     #s.push(1)
53     ],
54     s,
55     ),
56 )

```

Mixing animations and pause

Mixing animations and pause

Hello, there!

Mixing animations and pause

Hello, there!

Here are some switches you can choose.

Mixing animations and pause

Hello, there!

Here are some switches you can choose.



Mixing animations and pause

Hello, there!

Here are some switches you can choose.



If you press the green one, nothing
will happen.

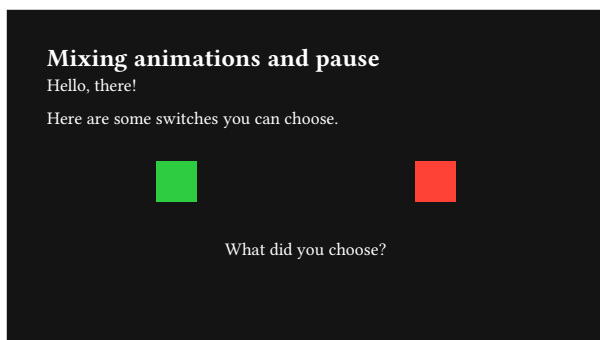
Mixing animations and pause

Hello, there!

Here are some switches you can choose.



If you press the red one, also
nothing...



Magic Selector Example

Integration with chemformula.

```

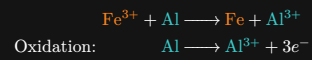
1  #import "@preview/sanor:0.3.0": *
2
3  // Set up presentation format.
4  #set page(paper: "presentation-16-9", fill: luma(20))
5  #set text(size: 25pt, fill: white)
6
7  #import "@preview/chemformula:0.1.3": ch
8
9  #slide(
10   defined-cases: ("grayed": text.with(fill: gray.transparentize(70%))),
11   s => (
12     [
13       #let tag = tag.with(s, hidden: none)
14       = Redox Reaction
15
16       #set align(center + horizon)
17       // Use selectors to apply styles on certain elements
18       #show select(ch("Fe")).or(select(ch("Fe^3+"))): set text(fill: orange)
19       #show select(ch("Al")).or(select(ch("Al^3+"))): set text(fill: teal)
20       #show select(ch("3e-")): tag.with("e-", hidden: "base")
21
22       $
23         && ch("Fe^3+ + Al &-> Fe + Al^3+") \
24         tag("ox", "Oxidation:&& ch("Al &-> Al^3+") + ch("3e-")) \
25         tag("red", "Reduction:&& ch("Fe^3+") + ch("3e-") ch("&-> Fe"))
26       $
27
28       #s.push(1)
29       #s.push(apply("ox"))
30       #s.push(apply("red"))
31       #s.push(apply("e-", text.with(fill: yellow)))
32       #s.push(force("e-", math.cancel, text.with(fill: luma(100))))
33     ],
34     s,
35   ),
36 )
37

```

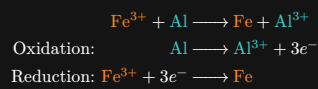
Redox Reaction



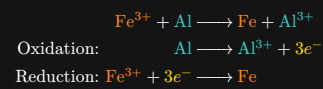
Redox Reaction



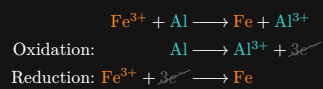
Redox Reaction



Redox Reaction



Redox Reaction



3 References

SANOR MODULE

- [pause\(\)](#)
- [scene\(\)](#)
- [select\(\)](#)
- [set-option\(\)](#)
- [slide\(\)](#)
- [tag\(\)](#)

pause

The `pause` function. Show the content after the current number of calls in `s`. Must be used with `#s.push(1)` for step forward, or `#s.push(-1)` for stepping backward.

```
1 #slide(s => ([
2   This is visible immediately.
3   #pause(s)[This appears after pushing a step.]
4   #s.push(1)
5 ], s))
```

Parameters

```
pause(  
  s: array ,  
  body: any ,  
  hidden: function case auto  
) -> any
```

s array

The slide animation sequence provided by `slide()`.

body any

The content to show or hide.

hidden function or case or auto

The case to use when content is hidden.

Default: auto

scene

A function for creating a slide, with an animation-context-included `tag` function.

Unlike the slide function, the animation context is already included in the `tag` callback. The rules for displaying the components must be specified in the `controls` argument instead.

Parameters

```
scene(  
  options: dictionary ,  
  func: function ,  
  hidden: auto case function ,  
  is-shown: bool ,  
  defined-cases: dictionary ,  
  controls: array  
) -> content
```

options dictionary

A configuration option defined to control the behavior of the slide.

Default: `(:)`

func function

A function that receives a `tag` function and returns content

hidden `auto` or `case` or `function`

Default modifier for tagged components in its hidden state.

Default: `auto`

is-shown `bool`

Whether to shown the tagged components by default when the component is not called by the control rules.

Default: `false`

defined-cases `dictionary`

A dictionary whose key is a name and value is a case to modify the tagged components.

Default: `(:)`

controls `array`

An array of rules that will be applied for each step of the animation

Default: `()`

select

Create a selector from content, a selector expression, or a matching value.

This helper is useful for `show` rules that need to target arbitrary content. It accepts strings, existing selectors, functions, regexes, symbols, labels, or content nodes. When passed content, it resolves the content's function and fields so that non-raw math elements can also be targeted.

Parameters

```
select(element: content) -> selector
```

element `content`

The thing to turn into a selector.

set-option

Sets global options for slides.

This function allows you to configure default options for all slides, such as enabling handout mode or defining default cases.

```
1 #let (slide,) = set-option(handout: true)
```

Parameters

```
set-option(  
  handout: bool ,  
  handout-index: auto int ,  
  ..new-options: dictionary  
)
```

handout bool

Handout mode

Default: false

handout-index auto or int

Index of the subslide used for handout slides. auto means using the last subslide.

Default: auto

..new-options dictionary

Named options to set globally for slides.

slide

The main slide function.

Creates an animated slide where content can be revealed or modified step by step. The function takes a slide context `s` and returns content. Use `s.push()` to add actions that control the animation sequence.

```
1 #slide(s => ([  
2   #let tag = tag.with(s)  
3   #tag("hello")[Hello from Sanor!]  
4   #s.push(apply("hello"))  
5 ], s))
```

Parameters

```
slide(  
  options: dictionary ,  
  func: function ,  
  hidden: case ,  
  is-shown: bool ,  
  defined-cases: dictionary  
)
```

options dictionary

The configuration option dictionary.

Default: `(:)`

func function

The body function. Must returns an array of `body` and modified sequence `s`. Usually written as `s => ([..], s)`.

hidden case

The default case applied to the hidden element.

Default: `auto`

is-shown bool

Whether to show the tagged elements by default.

Default: `false`

defined-cases dictionary

Reusable named cases for applying on the tagged elements. Must be in the form `name: case`

Default: `(:)`

tag

Tags content for animation control.

This function allows you to mark content that can be modified or revealed step by step during a presentation.

```
1 #slide(s => ([
2   #let tag = tag.with(s)
3   #tag("hello")[Hello World!]
4   #s.push(apply("hello", text.with(fill: red)))
5 ], s))
```

Parameters

```
tag(  
  s: array ,  
  name: str ,  
  body: any function ,  
  hidden: case function auto ,  
  ..defined-cases: arguments  
) -> any
```

s array

The slide animation sequence provided by `slide()`.

name str

A unique identifier for the tagged content.

body any or function

The content to tag. If a callback is provided, it will receive a non-hidden variant of `tag`, allowing nested elements to be tagged without automatically inheriting the hidden case.

hidden case or function or auto

The case to use when content is hidden.

Default: auto

..defined-cases arguments

Additional cases defined for this tag.

RULES MODULE

- [apply\(\)](#)
- [clear\(\)](#)
- [cover\(\)](#)
- [force\(\)](#)
- [once\(\)](#)
- [revert\(\)](#)

apply

Applies cases to tagged content for the current and all subsequent steps.

This function creates a rule that applies the specified cases to content tagged with the given name, and keeps those cases active in future steps.

Parameters

```
apply(  
  name: str,  
  ..cases: cases,  
  inherit: bool,  
  remain: bool,  
  active: bool  
) -> rule
```

name `str`

The tag name to apply to.

..cases `cases`

Cases or modifiers to apply.

inherit `bool`

Whether to combine with existing active cases.

Default: `true`

remain `bool`

Whether to remain the active cases to subsequent animations.

Default: `true`

active `bool`

Whether the animation can change the visibility of the element.

Default: `true`

`clear`

Clear the previous animation sequence on an element.

Parameters

```
clear(  
  name: str,  
  ..cases: cases  
) -> rule
```

name `str`

The tag name of the element to clear.

..cases `cases`

Cases or modifiers to apply.

cover

Covers tagged content by applying cases and preventing inheritance.

This function hides content by applying the specified cases without inheriting previous modifications.

Parameters

```
cover(  
  name: str,  
  ..cases: cases ,  
  inherit,  
  remain,  
  active  
) -> rule
```

name `str`

The tag name to apply to.

..cases `cases`

Cases or modifiers to apply.

force

Forces application of cases without inheritance.

This is equivalent to `apply(name, ..cases, inherited: false)`.

Parameters

```
force(  
  name: str,  
  ..cases: cases  
) -> rule
```

name `str`

The tag name to apply to.

..cases `cases`

Cases or modifiers to apply.

once

Applies cases to tagged content for only one step.

This function creates a rule that applies the specified cases to content tagged with the given name, but only for the current step.

Parameters

```
once(  
  name: str,  
  ..cases: cases ,  
  inherit,  
  remain,  
  active  
) -> rule
```

name `str`

The tag name to apply to.

..cases `cases`

Cases or modifiers to apply.

revert

Reverts tagged content to specified cases without inheritance. This keeps the previous modifications, unlike 'clear'

This function applies the specified cases to content without combining with previous modifications.

Parameters

```
revert(  
  name: str,  
  ..cases: cases ,  
  inherit,  
  remain,  
  active  
) -> rule
```

name `str`

The tag name to apply to.

..cases `cases`

Cases or modifiers to apply.

OBJECT-CASE MODULE

- [case\(\)](#)
- [object\(\)](#)

`case`

case definition

Creates a case that can modify content with stylers and wrappers. Stylers are named arguments that modify properties, wrappers are functions that transform the content.

Parameters

```
case(..modifiers: any) -> case
```

..modifiers `any`

Named stylers and positional wrapper functions.

`object`

Creates an object with different states.

This function creates a reusable object that can be displayed in different states defined by cases. The object can be called with different case names to apply various modifications.

Parameters

```
object(  
  func: function ,  
  hidden: case function ,  
  ..defined-cases: arguments  
) -> object
```

func `function`

The base function to create the object.

hidden `case` or `function`

The case to use when the object is hidden.

Default: `case(hide)`

..defined-cases arguments

Named cases defining different states.

COMPONENTS MODULE

- [ccomp\(\)](#)
- [mcomp\(\)](#)
- [new\(\)](#)

ccomp

CeTZ component constructor

Parameters

```
ccomp(  
  name: str,  
  func: function,  
  ..defined-cases: dictionary,  
  hidden: function case  
) -> object
```

name str

Default name of the component

func function

The component's function

..defined-cases dictionary

Named arguments of cases with their name as the key.

hidden function or case

Default hidden modifier that will apply when the component is in hidden case.

Default: `draw.hide.with(bounds: true)`

mcomp

Markup component constructor

Parameters

```
mcomp(  
  name: str,  
  func: function,  
  hidden: case function,  
  ..defined-cases: arguments  
) -> object
```

name str

Default name for the component

func function

Function to create the component

hidden case or function

Modifier to apply on the component in hidden case

Default: `component.case(hide)`

..defined-cases arguments

Keyword arguments of cases with their name as the key.

new

Component constructor

Helper for building reusable components. Components created with this constructor wrap an `object` and call `tag` for you. Note: the first argument passed at call-time (`tag-s`) may be either a slide context `s` or a `tag` function (for example the result of `tag.with(s)`). The constructor adapts automatically so components can be used both inside `slide` blocks and in helper contexts that already have a bound `tag`.

Parameters

```
new(  
  name: str,  
  func: function,  
  hidden: case function,  
  ..defined-cases: arguments  
) -> object
```

name str

Default name for the component

func `function`

Function to create the component

hidden `case` or `function`

Modifier to apply on the component in hidden case

Default: `case(hide)`

..defined-cases `arguments`

Keyword arguments of cases with their name as the key.

PDFPC MODULE

- [config\(\)](#)
- [speaker-note\(\)](#)

Variables

- [end-slide](#)
- [save-slide](#)
- [hidden-slide](#)

`config`

Configure presentation settings for pdfpc.

```
1 #pdfpc.config(  
2     duration-minutes: 30,  
3     start-time: "14:00",  
4     last-minutes: 5,  
5 )
```

Parameters

```
config(  
    duration-minutes: int ,  
    start-time: str datetime ,  
    end-time: str datetime ,  
    last-minutes: int ,  
    note-font-size: int ,  
    disable-markdown: bool ,  
    default-transition: dictionary  
)
```

duration-minutes `int`

Total presentation duration in minutes

Default: `none`

start-time `str` or `datetime`

Presentation start time in HH:MM format

Default: `none`

end-time `str` or `datetime`

Presentation end time in HH:MM format

Default: `none`

last-minutes `int`

Highlight final N minutes with visual alert

Default: `none`

note-font-size `int`

Font size of the speaker note

Default: `none`

disable-markdown `bool`

Whether or not to disable rendering the notes as markdown

Default: `false`

default-transition `dictionary`

Default slide transition settings

Default: `none`

speaker-note

Adds speaker notes to a slide, visible only in the presenter view.

```
1 #pdfpc.speaker-note[Remember to emphasize this point during the presentation]
```

Parameters

`speaker-note(text: str or raw) -> metadata`

text `str` or `raw`

Speaker notes as a string or raw code block

end-slide metadata

Mark the end of a slide

save-slide metadata

Save the current slide state

hidden-slide metadata

Mark a slide as hidden from the presentation